DocFlow AI - Backend README

1. Backend Overview

DocFlow AI backend is built using Django. It is organized into small apps:

```
apps/
■■■ accounts
■■■ organizations
■■■ departments
■■■ approvals
■■■ documents
■■■ audit
■■■ dashboard
■■■ superadmin
```

A separate FastAPI microservice is used for AI document analysis.

Core backend responsibilities:

- Custom user and profile model
- Organization scoped access
- Role based dashboards
- Department management
- Approval request CRUD
- Approval workflow templates
- Workflow step generation
- Approve, reject, comment actions
- Document upload and PDF text extraction
- Audit logging
- Super Admin platform management
- AI microservice integration

2. Main Technology Stack

```
Python
Django
PostgreSQL
Bootstrap templates
WhiteNoise static serving
Django REST Framework
Django Filter
Celery
Redis
FastAPI AI service
PyMuPDF for PDF text extraction
```

3. Django Apps

3.1 Accounts App

Path:

```
apps/accounts/
```

Responsibilities:

- Custom `User` model
- `UserProfile` model
- User roles
- Organization registration
- Normal login
- Super Admin separate login
- Logout
- Profile page
- Role based dashboard redirect
- Role based decorators

Important role enum:

```
ORG_ADMIN
DEPARTMENT_HEAD
STAFF
APPROVER
AUDITOR
```

Super Admin is handled with user.is_superuser.

Important files:

```
models.py
forms.py
views.py
urls.py
utils.py
decorators.py
signals.py
admin.py
```

3.2 Organizations App

Path:

```
apps/organizations/
```

Responsibilities:

- Organization workspace model
- Slug generation
- Admin registration
- Logo storage

Important model fields:

```
name
slug
email
phone
address
logo
is_active
created_at
updated_at
```

Organization is the root tenant scope for the SaaS.

3.3 Departments App

Path:

```
apps/departments/
```

Responsibilities:

- Organization scoped department CRUD
- Department head assignment
- Department users list
- Audit logging for department create/update/delete

Important rules:

- Only Organization Admin can create/update/delete department.
- Super Admin can view departments but should not create department directly because organization is required.
- Department names must be unique inside the same organization.
- Assigning a department head updates the selected profile role and department.

3.4 Approvals App

Path:

```
apps/approvals/
```

Responsibilities:

- Approval requests
- Request filters/search
- Workflow templates
- Workflow steps
- Submit request
- Generate approval steps
- Approval inbox
- Approve/reject/comment
- Request timeline

Important models:

```
DocumentRequest
ApprovalWorkflow
ApprovalWorkflowStep
ApprovalStep
ApprovalAction
```

Important enums:

```
RequestStatus
RequestPriority
RequestCategory
RiskLevel
ApprovalStepStatus
ApprovalActionType
```

Important services:

```
services/workflow_engine.py
services/action_engine.py
```

3.5 Documents App

Path:

```
apps/documents/
```

Responsibilities:

- Multiple file upload
- File validation
- PDF text extraction
- Attachment detail
- Attachment delete
- AI placeholder fields
- Audit logging for upload/delete

Important model:

```
DocumentAttachment
```

Important fields:

```
request
uploaded_by
file
original_filename
file_type
file_size
extracted_text
ai_summary
ai_category
ai_extracted_data
risk_level
risk_notes
ai_processed
processing_error
created_at
updated_at
```

Allowed extensions:

```
.pdf
.jpg
.jpeg
.png
.doc
.docx
```

3.6 Audit App

Path:

```
apps/audit/
```

Responsibilities:

- AuditLog model
- Audit action enum
- IP capture
- User-agent capture
- Reusable audit logger
- Audit log list/detail pages
- Organization scoped logs

Important helper:

```
apps/audit/services/audit_logger.py
```

Important action examples:

```
CREATED
UPDATED
DELETED
SUBMITTED
CANCELLED
APPROVED
REJECTED
COMMENTED
UPLOADED
STATUS_CHANGED
```

3.7 Dashboard App

Path:

```
apps/dashboard/
```

Responsibilities:

- Role based dashboard home redirect
- Super Admin dashboard
- Organization Admin dashboard
- Department Head dashboard

- Staff dashboard
- Approver dashboard
- Auditor dashboard

No database models are required for dashboard app.

3.8 Superadmin App

Path:

```
apps/superadmin/
```

Responsibilities:

- Super Admin custom organization management
- Organization detail
- Organization activate/deactivate
- Platform users list
- User activate/deactivate
- Audit logs for status changes

No database models are required in this app.

4. Request Scope Rules

Use organization scoped querysets everywhere.

Basic logic:

```
Super Admin -> sees all
Org Admin -> sees own organization
Department Head -> sees own department
Staff -> sees own created requests
Approver -> sees assigned requests/steps
Auditor -> sees organization requests/logs
```

Never show data from another organization to normal users.

5. Approval Workflow Engine

Workflow template contains ordered steps.

When request is submitted:

1. Request status changes from DRAFT to SUBMITTED.
2. Workflow steps are converted to generated ApprovalStep records.
3. First generated step becomes current.
4. Request moves to IN_REVIEW.
5. ApprovalAction is created for submit.

6. AuditLog is created.

Assignment priority:

```
1. Fixed approver user
2. Department based assignment
3. User with required role
4. Organization Admin fallback
```

6. Approval Action Engine

Approver can:

- Add comment
- Approve current step
- Reject current step

Approve logic:

```
Current step -> APPROVED
If next pending step exists -> next step becomes current, request remains IN_REVIEW
If no next step -> request becomes APPROVED
```

Reject logic:

```
Current step -> REJECTED
Remaining pending steps -> SKIPPED
Request -> REJECTED
```

Comment logic:

```
Append comment to current step
Create ApprovalAction COMMENTED
Create AuditLog COMMENTED
```

7. Audit Logging Strategy

Use this helper everywhere:

```
create_audit_log(
request=request,
organization=organization,
action=AuditAction.CREATED,
model_name="DocumentRequest",
object_id=obj.pk,
message="Human readable message",
extra_data={...},
)
```

Events already logged:

```
Request create
Request update
Request submit
Request cancel
Request delete
Approval approve
Approval reject
Approval comment
Document upload
```

```
Document delete
Department create/update/delete
Organization activate/deactivate
User activate/deactivate
```

8. FastAPI AI Service

The AI service runs separately on:

```
http://localhost:8001
```

Main endpoint:

```
POST /ai/analyze-document/
```

Input:

```
{
"text": "Invoice amount Rs. 25000 dated 12/05/2026 approved and signed.",
"filename": "invoice.pdf"
}
```

Output:

```
{
"success": true,
"summary": "...",
"category": "VENDOR_PAYMENT",
"extracted_fields": {},
"missing_fields": [],
"risk_level": "LOW",
"risk_notes": "No major risk found."
}
```

9. Important Settings

In config/settings/base.py:

```
AUTH_USER_MODEL = "accounts.User"
TIME_ZONE = "Asia/Kolkata"
STATIC_ROOT = BASE_DIR / "staticfiles"
MEDIA_ROOT = BASE_DIR / "media"
AI_SERVICE_URL = "http://localhost:8001"
MAX_UPLOAD_SIZE = 10 * 1024 * 1024
```

10. Backend Commands

```
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py collectstatic
python manage.py runserver
```

FastAPI:

```
uvicorn main:app --reload --host 127.0.0.1 --port 8001
```

11. Backend Testing Checklist

- Register organization
- Login as organization admin
- Create departments
- Create workflow
- Add workflow steps
- Create request
- Upload document
- Submit request
- Login as approver
- Approve/reject/comment
- Check request timeline
- Check audit logs
- Check Super Admin dashboard
- Activate/deactivate organization
- Activate/deactivate user